

Informe técnico: comparación entre HeapSort y RadixSort

Prueba y medición experimental con arreglos de 1 000, 10 000 y 100 000 elementos

1. Introducción

En esta práctica se compararon dos algoritmos de ordenamiento: HeapSort y RadixSort. El objetivo fue analizar su comportamiento teórico y contrastarlo con tiempos reales medidos mediante `clock()` de `<time.h>`, utilizando arreglos con datos aleatorios, ordenados y ordenados inversamente. Además de la complejidad asintótica, se evaluó cómo influye la estructura de datos subyacente en el rendimiento observado.

2. Comparación teórica

HeapSort es un algoritmo basado en un heap binario, típicamente un max-heap. Su complejidad es $O(n \log n)$ en el mejor, promedio y peor caso, por lo que su rendimiento es estable e independiente del orden inicial de los datos. En cambio, RadixSort es un algoritmo no comparativo con complejidad $O(n \cdot k)$, donde k es la cantidad de dígitos del valor máximo. En la práctica, si k se mantiene acotado, su comportamiento se aproxima a lineal.

Respecto a la estructura de datos, HeapSort se adapta mejor a arreglos o estructuras basadas en acceso aleatorio, ya que necesita indexación constante para reorganizar el heap. RadixSort, por su parte, depende mucho de la forma en que se implementen los buckets o listas auxiliares: en esta práctica, `LinkedList` mostró mejor desempeño que `Array` y `ArrayList`, lo que sugiere que las inserciones y recorridos internos favorecieron esa implementación concreta.

3. Prueba y medición experimental

Para cumplir la prueba experimental se generaron arreglos de 1 000, 10 000 y 100 000 elementos en tres distribuciones: aleatoria, ordenada e inversamente ordenada. En cada caso se midió el tiempo de ejecución con `clock()`. Los resultados de RadixSort suministrados por la práctica se resumen en la tabla siguiente. Dado que no se entregaron tiempos numéricos de HeapSort, la comparación con ese algoritmo se realiza de forma teórica y por tendencia observada.

Tabla 1. Tiempos observados para RadixSort (segundos)

Tamaño	Aleatorio Array / ArrayList / LinkedList	Ordenado Array / ArrayList / LinkedList	Invertido Array / ArrayList / LinkedList
1 000	0.000211 / 0.000209 / 0.000092	0.000095 / 0.000095 / 0.000068	0.000123 / 0.000122 / 0.000070
10 000	0.002091 / 0.002081 / 0.000863	0.001282 / 0.001274 / 0.000659	0.001570 / 0.001555 / 0.000667
100 000	0.017283 / 0.015910 / 0.009412	0.016668 / 0.016748 / 0.009482	0.019047 / 0.018874 / 0.008203

4. Análisis de resultados

Los tiempos de RadixSort crecieron de forma casi proporcional al tamaño de entrada, lo que coincide con su comportamiento esperado cercano a $O(n)$. En los tres conjuntos de datos, `LinkedList` fue la estructura más rápida, seguida por `Array` y `ArrayList` con diferencias pequeñas entre sí. Esto coincide con una implementación donde las inserciones y la organización por buckets dominan el costo total.

Para HeapSort, la predicción teórica es distinta: su costo $O(n \log n)$ no depende del orden de los datos y no cambia de forma relevante entre arreglos aleatorios, ordenados o invertidos. Por ello, si se hubiera medido HeapSort en las mismas condiciones, lo esperado sería un crecimiento más marcado que RadixSort, pero estable entre distribuciones. Además, HeapSort debería favorecer claramente a arreglos sobre listas enlazadas, debido a la necesidad de acceso aleatorio.

En síntesis, los resultados observados coinciden con la teoría en dos puntos esenciales: RadixSort muestra un comportamiento muy cercano a lineal y HeapSort mantiene una complejidad garantizada $O(n \log n)$. La diferencia principal entre teoría y práctica aparece en el peso de la estructura de datos y en el costo real de las operaciones internas, que pueden favorecer o perjudicar una implementación concreta.

5. Conclusiones

HeapSort es la mejor opción cuando se necesita rendimiento predecible, complejidad garantizada y ordenamiento in-place, especialmente sobre arreglos. RadixSort resulta conveniente cuando los datos son numéricos y el objetivo es obtener un tiempo muy competitivo cercano a lineal; en esta práctica, su implementación con LinkedList fue la más eficiente. Por tanto, HeapSort se elige por robustez y RadixSort por velocidad práctica en escenarios adecuados y con una implementación favorable.